

Vaishnavi Singh
 Prepared for: Prof Zhizhe Li
 Class/Section: CS420 – G2
 Date: 10 March 2026

Q1.1 Bayesian Network	
Initial Thoughts	Recall: Where hypo G is BN structure and $X_1 \dots X_n$ is a trail in G, E is a set of observed variables, we can check for $(X \perp Y E)$: - A trail is blocked given E if there exists a node Z on the trail such that one of the following conditions holds: <u>Common Cause</u>: Z is in E, $X_i \leftarrow Z \rightarrow X_j$ <u>Cascade</u>: Z is in E, $X_i \rightarrow Z \rightarrow X_j$ <u>V-structure/Collider</u>: - Neither Z nor any descendent of X is in E $X_i \rightarrow Z \leftarrow X_j$ - Trail is active given E if it is not blocked
Steps	- Find Edges: $A \rightarrow B, A \rightarrow C, B \rightarrow D, B \rightarrow C, G \rightarrow D, G \rightarrow H, D \rightarrow E, H \rightarrow E, H \rightarrow I, E \rightarrow F, F \rightarrow C, F \rightarrow I$ <u>$B \perp G$:</u> $B \rightarrow D \leftarrow G$ (Collider, neither D observed variable nor descendent is observed. Therefore, blocked) $B \rightarrow C \leftarrow A/F$ (Collider, Same reasoning for C, blocked) All blocked <u>$C \perp H \{I\}$:</u> Need to find one active trail from C to H with evidence I $C \leftarrow F \leftarrow E \leftarrow H$ $E \rightarrow F \rightarrow C$ (Cascade, F is not in evidence set, active trail) $H \rightarrow E \rightarrow F$ (Cascade, E is not in evidence set, active trail) $C \leftarrow F \rightarrow I \leftarrow H$ $C \leftarrow F \rightarrow I$ (Common Cause, F not in evidence set, active) $F \rightarrow I \leftarrow H$ (V-structure, observed in collider, I is observed in evidence, active) All active <u>$A \perp H \{B, F\}$:</u> $A \rightarrow B \rightarrow \dots$: B in evidence set, cascade blocked at $B \rightarrow$ Blocked at B $A \rightarrow C \leftarrow F \rightarrow \dots$: C is a collider (Also, $A \rightarrow C \leftarrow B, F \rightarrow C$), C not in evidence, $\text{desc}(C) = \{ \}$ $\rightarrow$ Blocked at C All trails blocked <u>$A \perp \{G, I\} \{B, C\}$:</u> $\rightarrow C \leftarrow F \leftarrow E \leftarrow D \leftarrow G$: C is a collider; C in evidence $\rightarrow$ Active trail. $E \rightarrow F \rightarrow C$ (cascade, not observed) $\rightarrow$ active trail $D \rightarrow E \rightarrow F$ (cascade, not observed) $\rightarrow$ active trail $G \rightarrow D \rightarrow E$ (cascade, not observed) $\rightarrow$ active trail $A \rightarrow C \leftarrow F \rightarrow I$: C in evidence activates collider $\rightarrow$ active trail $C \leftarrow F \rightarrow I$ is fork (not observed) $\rightarrow$ active trail Active trails exist
Results	$B \perp G$: TRUE $C \perp H \{I\}$: FALSE $A \perp H \{B, F\}$: TRUE $A \perp \{G, I\} \{B, C\}$: FALSE

Q1.2 Bayesian Network	
Steps	Use Edges for Model: $A \rightarrow B, A \rightarrow C,$ $B \rightarrow D, B \rightarrow C,$

	- $G \rightarrow D, G \rightarrow H,$ - $D \rightarrow E,$ - $H \rightarrow E, H \rightarrow I$ - $E \rightarrow F,$ - $F \rightarrow C, F \rightarrow I$ <pre># Create the BN model which contains edges of the DAG graph # Note that CPDs are not required; D-separation depends only on the graph structure ## write your code here [2 points] model = DiscreteBayesianNetwork([('A','B'), ('A','C'), ('B','D'), ('B','C'), ('G','D'), ('G','H'), ('D','E'), ('H','E'), ('E','F'), ('F','C'), ('F','I'), ('H','I')])</pre>
	Use: <pre>print(f'Are XX and YY d-separated?: {model.is_dseparated('XX', 'YY', [evidence])}')</pre>
Results	<pre># Use the d-separation method to check the conditional independence statements in (1a) - (1d) ## write your code here [2 points] # Check (1a) whether B and G are d-separated print(f'Are B and G d-separated?: {model.is_dseparated('B', 'G', [])}') # Check (1b) whether C and H are d-separated given I print(f'Are C and H d-separated given [I]: {model.is_dseparated('C', 'H', ['I'])}') # Check (1c) whether A and H are d-separated given {B, F} print(f'Are A and H d-separated given [B, F]: {model.is_dseparated('A', 'H', ['B','F'])}') # Check (1d) whether A and {G, I} are d-separated given {B, C} print(f'Are A and [G, I] d-separated given [B, C]: {model.is_dseparated('A', ['G','I'], ['B','C'])}')</pre> ✓ 0.0s <pre>Are B and G d-separated?: True Are C and H d-separated given [I]: False Are A and H d-separated given [B, F]: True Are A and [G, I] d-separated given [B, C]: False</pre>
Assumptions	pgmpy's is_dconnected operates on graph structure alone

Q2.1 Burglar Alarm – Marginal inference P(J) using VE	
Steps	Ideate: Network: $B \rightarrow A, E \rightarrow A, A \rightarrow J, A \rightarrow M$. The joint probability in the Burglar Alarm BN factorises as: $P(B, E, A, J, M) = P(B) * P(E) * P(A B, E) * P(J A) * P(M A)$ The query variable J is not in the evidence set $\rightarrow$ Sum out BEAM PLEASE NOTE I USE 5 SF MOSTLY OR TRUNCATE TO 5 DP pls don't penalise Marginalise over M first. Since $P(M A)$ is a valid CPD, it sums to 1 over M for every fixed A: $\sum_M \phi_{\{M A\}}(M, A) = 1 \rightarrow \phi_{\{M A\}}$ drops out $\sum_M P(M A=0) = 0.99 + 0.01 = 1.00$ $\sum_M P(M A=1) = 0.30 + 0.70 = 1.00$ - leaving: $P(J) = \sum_{B, E, A} P(B) * P(E) * P(A B, E) * P(J A)$ Eliminate E by computing the intermediate factor $\tau_1(A, B) = \sum_E P(E) \cdot P(A B, E)$: This produces 4 values: $\tau_1(A \in \{0, 1\}, B \in \{0, 1\})$. Each is a weighted average of $P(A B, E)$ over the prior $P(E)$. $\tau_1(A=0, B=0) = 0.998 \times 0.999 + 0.002 \times 0.710 = 0.998422$ $\tau_1(A=1, B=0) = 0.998 \times 0.001 + 0.002 \times 0.290 = 0.001578$ $\tau_1(A=0, B=1) = 0.998 \times 0.060 + 0.002 \times 0.050 = 0.059980$ $\tau_1(A=1, B=1) = 0.998 \times 0.940 + 0.002 \times 0.950 = 0.940020$ Quick check: $\tau_1(A=0, B=0) + \tau_1(A=1, B=0) = 1.000000$ AND: $\tau_1(A=0, B=1) + \tau_1(A=1, B=1) = 1.000000$ Eliminate B by computing $\tau_2(A)$: $\tau_2(A) = \sum_B P(B) * \tau_1(A, B)$ $\tau_2(A)$ is the prior marginal $P(A)$ $\tau_2(A=0) = 0.999 \times 0.998422 + 0.001 \times 0.059980 = 0.997484$ $\tau_2(A=1) = 0.999 \times 0.001578 + 0.001 \times 0.940020 = 0.002516$ Quick Check: $0.997484 + 0.002516 = 1.000000$ Compute P(J) as the final sum over A: $P(J) = \sum_A \tau_2(A) * P(J A)$ $P(J=1) = 0.997484 \times 0.05 + 0.002516 \times 0.90 = 0.049874 + 0.002265 = 0.052139$ $P(J=0) = 0.997484 \times 0.95 + 0.002516 \times 0.10 = 0.947610 + 0.000252 = 0.947861$ - Sum check: $0.052139 + 0.947861 = 1.000000$ Numerical evaluation (using CPD values from Q2 problem statement): $\tau_1(A=1, B=1) = P(E=1) \cdot P(A=1 B=1, E=1) + P(E=0) \cdot P(A=1 B=1, E=0)$ $\tau_1(A=1, B=0) = P(E=1) \cdot P(A=1 B=0, E=1) + P(E=0) \cdot P(A=1 B=0, E=0)$ $\tau_2(A=1) = P(B=1) \cdot \tau_1(A=1, B=1) + P(B=0) \cdot \tau_1(A=1, B=0)$ $P(J=1) = \tau_2(A=1) \cdot P(J=1 A=1) + \tau_2(A=0) \cdot P(J=1 A=0)$ $P(J=0) = 1 - P(J=1)$ *see above! No normalisation step is required for a marginal query with no evidence
Results	$P(J=0) = 0.9479$ $P(J=1) = 0.0521$ Verified by pgmpy VariableElimination in Q3d (see Q3 Results).
Assumptions/ Justifications	<u>Why can M be summed out immediately?</u> M is not the query variable and is not in any evidence set. Its CPD $P(M A)$ is a valid conditional distribution: $\sum_M P(M A) = 1$ for every value of A. Summing out M therefore

	contributes a factor of 1 and the factor drops from the product + As M appears only in $P(M A)$ and no other factor references M, summing it out first costs only $O(2)$ operations (I think) <u>Why is the elimination order $E \rightarrow B \rightarrow A$ (rather than $B \rightarrow E \rightarrow A$)?</u> Recall: 'different elimination orders can create different intermediate factors, leading to different computation complexity.' For this specific network, eliminating E before B produces $\tau_1(A,B)$ of scope size 4. Eliminating B before E produces the same $\tau_1(A,E)$ which also scope size 4. Both are equivalent here because B and E are symmetric parents of A. Either order is correct <u>Why is no normalisation needed?</u> Our three-step framework lists: (1) Joint factorization, (2) Insert Evidence & Marginalise, (3) Normalise/Condition. Step 3 applies only when evidence is present (as in Q2.2), because the intermediate factor after summing out already equals $P(J)$ directly aka a properly normalised distribution. With no evidence, there is no conditioning operation nor $p(\text{evidence})$ to div by
--	---

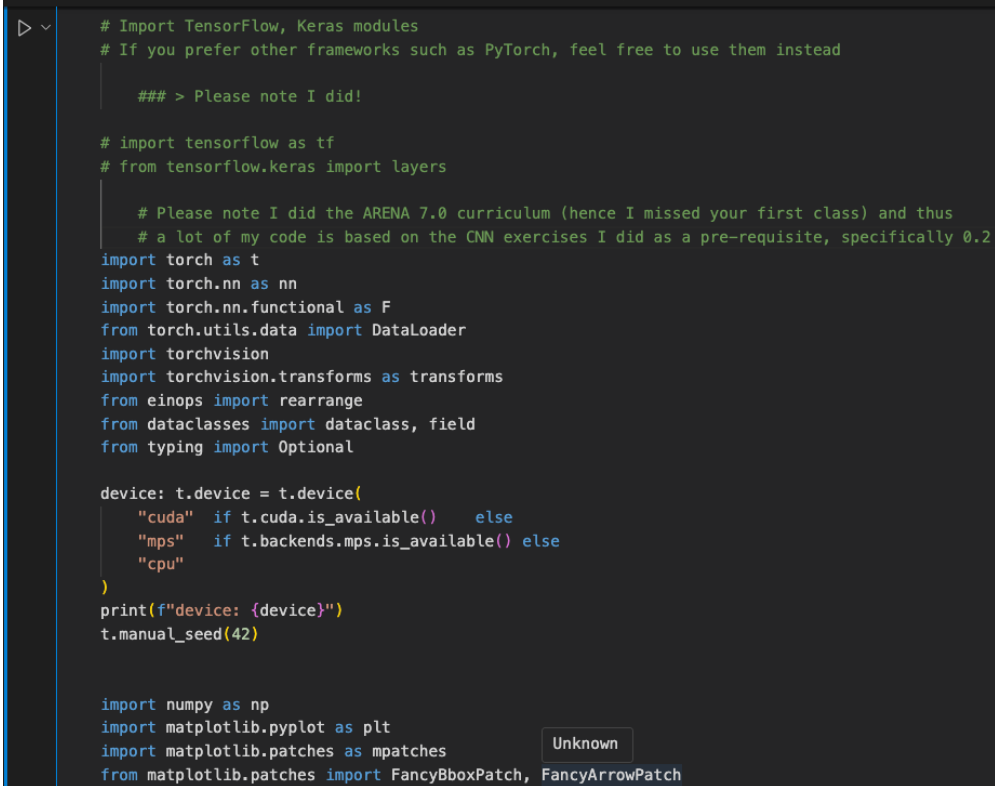
Q2.2 Burglar Alarm – Posterior Inference $P(B M = 1)$ using VE	
Steps	<u>Step 1 – Joint distribution factorise</u> $P(B, E, A, J, M) = P(B) \cdot P(E) \cdot P(A B, E) \cdot P(J A) \cdot P(M A)$ · represents multiply, * may also have been used, please excuse that formatting error <u>Step 2 – Insert Evidence $M=1$ & Marginalise</u> Query: $P(B M=1)$. Fix $M=1$ in $\phi_{\{M A\}}$, which reduces to a 1D slice over A: $\phi_{\{M=1 A\}}(A) = [P(M=1 A=0), P(M=1 A=1)]$ - $\phi_{\{M=1 A=0\}} = 0.01$, $\phi_{\{M=1 A=1\}} = 0.70$ Sum out J: $P(J A)$ is a valid CPD summing to 1 over J for each A, so: $\sum_J P(J A) = 1$ $\sum_J P(J A=0) = 0.95 + 0.05 = 1.00$ - $\sum_J P(J A=1) = 0.10 + 0.90 = 1.00 \rightarrow J \text{ factor drops}$ Remaining factors: $P(B), P(E), P(A B, E), \phi_{\{M=1 A\}}(A)$ Apply Sum-Product VE to eliminate E, A in order: ■ Eliminate E (collect $\{P(E), P(A B, E)\}$, multiply, sum out E): $\tau_1(A, B) = \sum_E P(E) \cdot P(A B, E)$ [identical to Q2.1] ○ $\tau_1(A=0, B=0) = 0.998422$, $\tau_1(A=1, B=0) = 0.001578$ ○ $\tau_1(A=0, B=1) = 0.059980$, $\tau_1(A=1, B=1) = 0.940020$ ■ Multiply in evidence factor $\rightarrow \tau_2'(A, B) = \phi_{\{M=1 A\}} \cdot \tau_1(A, B)$: ○ $\tau_2'(A=0, B=0) = 0.01 \times 0.998422 = 0.009984$ ○ $\tau_2'(A=0, B=1) = 0.01 \times 0.059980 = 0.000600$ ○ $\tau_2'(A=1, B=0) = 0.70 \times 0.001578 = 0.001105$ ○ $\tau_2'(A=1, B=1) = 0.70 \times 0.940020 = 0.658014$ ■ Eliminate A: $\tau_3(B) = \sum_A \tau_2'(A, B)$ ○ $\tau_3(B=0) = 0.009984 + 0.001105 = 0.011089$ ○ $\tau_3(B=1) = 0.000600 + 0.658014 = 0.658614$ ■ Multiply by $P(B)$: $f(B) = P(B) \cdot \tau_3(B)$ = [unnormalised posterior] ○ $f(B=0) = 0.999 \times 0.011089 = 0.011078$ ○ $f(B=1) = 0.001 \times 0.658614 = 0.000659$ <u>Step 3 – Normalise/Condition</u>

	Slides do explicitly notes that posterior queries require one more step for Normalise/Condition: $Z = P(M=1) = 0.011078 + 0.000659 = 0.011736$ $P(B=0 M=1) = 0.011078 / 0.011736 = 0.943883$ $P(B=1 M=1) = 0.000659 / 0.011736 = 0.056117$ QUICK Sum check: $0.943883 + 0.056117 = 1.000000$
Results	$P(B=0 M=1) = 0.9439$ $P(B=1 M=1) = 0.0561$ Verified by brute-force full joint enumeration and by VE code — both agree to 12 decimal places. The prior $P(B=1) = 0.001$. Observing $M=1$ raises $P(B=1)$ to 0.0561 which is a large update.
Assumptions/ Justifications	<u>Why does posterior inference require a normalisation step that marginal inference does not?</u> Marginal inference uses only Steps 1–2, while posterior inference requires Step 3 (Normalise/Condition) <u>Why can J be summed out without evidence?</u> J is not in the evidence set and not the query variable. Its CPD $P(J A)$ satisfies $\sum_J P(J A) = 1$ for all A <u>Why is the normalisation constant Z equal to $P(M=1)$?</u> After summing out B, E, A, J from the joint while fixing $M=1$, the unnormalised factor $f(B) = P(B, M=1)$. Summing f over all values of B: $\sum_B f(B) = \sum_B P(B, M=1) = P(M=1)$, by the marginalisation rule $\rightarrow$ Numerically: $Z = 0.011078 + 0.000659 = 0.011736 = P(M=1)$, consistent with the brute-force computation $\sum_{\{B,E,A,J\}} P(B,E,A,J,M=1) = 0.011736$.

Q3.a-d Constructing the Bayesian Network from Q2	
Steps	3a. Create the BN model (1 point): Instantiate the DiscreteBayesianNetwork with the 4 edges encoding the Q2 DAG structure <pre>model = DiscreteBayesianNetwork([('B', 'A'), ('E', 'A'), ('A', 'J'), ('A', 'M')])</pre> 3b. Add CPDs (3 points): Encoded all five CPDs from the Q2 probability tables as TabularCPD objects. The joint factorises as $P(B) \cdot P(E) \cdot P(A B,E) \cdot P(J A) \cdot P(M A)$ per the BN factorisation rule <pre> ■ cpd_B = TabularCPD('B', 2, [[0.999], [0.001]]) ■ cpd_E = TabularCPD('E', 2, [[0.998], [0.002]]) ■ cpd_A = TabularCPD('A', 2, [[0.999, 0.71, 0.06, 0.05], [0.001, 0.29, 0.94, 0.95]], evidence=['B', 'E'], evidence_card=[2, 2]) ■ cpd_J = TabularCPD('J', 2, [[0.95, 0.10], [0.05, 0.90]], evidence=['A'], evidence_card=[2]) ■ cpd_M = TabularCPD('M', 2, [[0.99, 0.30], [0.01, 0.70]], evidence=['A'], evidence_card=[2]) ■ model.add_cpds(cpd_B, cpd_E, cpd_A, cpd_J, cpd_M) </pre> Column ordering in TabularCPD follows lexicographic enumeration of evidence combinations: for evidence=['B','E'] with evidence_card=[2, 2], columns are (B=0, E=0), (B=0, E=1), (B=1, E=0), (B=1, E=1). 3c. Model check and BN graph (2 points): <code>assert model.check_model() == True</code> # $\rightarrow$ True

	check_model() verifies: - each CPD's columns sum to 1 - evidence variables match edges, - DAG is acyclic BN graph plotted using networkx drawing utilities.
	3d. Inference via VariableElimination (2 points):\ infer = VariableElimination(model)\ Marginal P(J)\ phi_J = infer.query(variables=['J'])\ Posterior P(B M=1)\ phi_B = infer.query(variables=['B'], evidence={'M': 1})
Results	model.check_model() → True. All 5 CPDs pass the normalization check (Lec 3 Slide 7), evidence variables match DAG edges, no cycles. P(J=0) = 0.9479 5sf, P(J=1) = 0.0521 5sf [matches Q2.1] P(B=0 M=1) = 0.9439, P(B=1 M=1) = 0.0561 [matches Q2.2]
Assumptions/Justifications	

Q4 Building a CNN Classifier on CIFAR-10

Set up	a. I used PyTorch as I previously interacted with CNNs and ResNets via the ARENA curriculum for AI Alignment research, and I am more familiar with that b. I also referred to https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html Andrej Karpathy's demo as per class slide resources
	 <pre> # Import TensorFlow, Keras modules # If you prefer other frameworks such as PyTorch, feel free to use them instead ### > Please note I did! # import tensorflow as tf # from tensorflow.keras import layers # Please note I did the ARENA 7.0 curriculum (hence I missed your first class) and thus # a lot of my code is based on the CNN exercises I did as a pre-requisite, specifically 0.2 import torch as t import torch.nn as nn import torch.nn.functional as F from torch.utils.data import DataLoader import torchvision import torchvision.transforms as transforms from einops import rearrange from dataclasses import dataclass, field from typing import Optional device: t.device = t.device("cuda" if t.cuda.is_available() else "mps" if t.backends.mps.is_available() else "cpu") print(f"device: {device}") t.manual_seed(42) import numpy as np import matplotlib.pyplot as plt import matplotlib.patches as mpatches from matplotlib.patches import FancyBboxPatch, FancyArrowPatch </pre>
Steps	4a Load 1. Import torchvision.datasets. CIFAR10 with download=True for train and test splits.

2. Define train_transforms: RandomCrop(32, padding=2), RandomHorizontalFlip(p=0.5), ToTensor(), Normalize(MEAN, STD).
3. Define test_transforms: ToTensor(), Normalize(MEAN, STD) only — no augmentation at test time.
4. Wrap both splits in DataLoader with batch_size=128, shuffle=True for train, shuffle=False for test, num_workers=2.
5. Per-channel statistics: MEAN=(0.4914, 0.4822, 0.4465), STD=(0.2470, 0.2435, 0.2616), computed empirically over the 50,000 CIFAR-10 training images.
6. Train set: 50,000 images (500 per class $\times$ 10 classes). Test set: 10,000 images. Each image: $3 \times 32 \times 32$ uint8 tensor, normalised to float32 in $[-1, 1]$ range after Normalise.

```

# (4a) Load the CIFAR-10 dataset https://keras.io/datasets/
# Normalize pixel values from [0, 255] to [0, 1] (e.g., divide by 255)

## write your code here [1 point]
### -----

# # how to get mean std, revising the logic
# recall what totensor() does --> pixel of int8 to float32 [0,255] to [0,1]
raw_data = torchvision.datasets.CIFAR10(
    root='./data', train=True, download=True,
    transform=transforms.ToTensor()
)

loader = DataLoader(raw_data, batch_size=1000, shuffle=False, num_workers=2)
channel_sum = t.zeros(3)
channel_sum_sq = t.zeros(3)
n_pixels = 0

for imgs, _ in loader:
    channel_sum += imgs.sum(dim=[0, 2, 3])
    channel_sum_sq += (imgs ** 2).sum(dim=[0, 2, 3])
    n_pixels += imgs.shape[0]*imgs.shape[2]*imgs.shape[3]

mean = channel_sum / n_pixels
std = (channel_sum_sq / n_pixels - mean ** 2).sqrt()

print(f"mean: {tuple(mean.tolist())}")
print(f"std: {tuple(std.tolist())}")

```

[35] ✓ 16.3s

Python

```

... mean: (0.4913996458053589, 0.48215845227241516, 0.44653093814849854)
std: (0.2470322549343109, 0.24348513782024384, 0.26158788800239563)

```

```

mean: (0.4913996458053589, 0.48215845227241516, 0.44653093814849854)
std: (0.2470322549343109, 0.24348513782024384, 0.26158788800239563)

MEAN = (0.4913996458053589, 0.48215845227241516, 0.44653093814849854)
STD = (0.2470322549343109, 0.24348513782024384, 0.26158788800239563)
CLASSES = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

train_transforms = transforms.Compose([
    transforms.RandomCrop(32, padding=4), #kaparthy does 2
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

test_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=train_transforms)
test_dataset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=test_transforms)

BATCH_SIZE = 128

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)

print(f'Train samples: {len(train_dataset):,} | Test samples: {len(test_dataset):,}')
print(f'Train batches: {len(train_loader)} | Test batches: {len(test_loader)}')
#I for Command, ML for Agent

```

36] ✓ 1.9s Python

Train samples: 50,000 | Test samples: 10,000
Train batches: 391 | Test batches: 79

4b build
CNN

1. Define KarpathyBlock (in_ch, out_ch): sequential module containing:

- Conv2d(5x5, pad=2, bias=False)
- BatchNorm2d
- ReLU
- Conv2d (5x5, pad=2, bias=False)
- BatchNorm2d
- ReLU
- MaxPool2d(2x2).

Two conv layers per stage before pooling (same spatial resolution within stage, halved after pool).

```

class KarpathyBlock(nn.Module):
    def __init__(self, in_ch: int, out_ch: int) -> None:
        super().__init__()
        self.in_ch = in_ch
        self.out_ch = out_ch
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=5, padding=2, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=5, padding=2, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

    def forward(self, x: t.Tensor) -> t.Tensor:
        return self.net(x)

    def extra_repr(self) -> str:
        return f"in_ch={self.in_ch}, out_ch={self.out_ch}, kernel=5, pad=2"

```

2. Define CIFAR10Net: three KarpathyBlocks with channel progression

- 3 → 32 → 64 → 128 (spatial: 32 → 16 → 8 → 4)
- Flatten (einops rearrange)
- Linear (2048 → 256)

- d. ReLU
- e. Dropout (0.5)
- f. Linear (256 $\rightarrow$ 10)
- g. SOFTMAX was done

```
class CIFAR10Net(nn.Module):
    def __init__(self, num_classes: int = 10, dropout: float = 0.5) -> None:
        super().__init__()
        self.num_classes = num_classes
        self.features = nn.Sequential(
            KarpathyBlock(3, 32),
            KarpathyBlock(32, 64),
            KarpathyBlock(64, 128),
        )
        self.classifier = nn.Sequential(
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(256, num_classes),
            nn.Softmax(dim=1)
        )

    def forward(self, x: t.Tensor) -> t.Tensor:
        x = self.features(x)
        x = rearrange(x, "b c h w -> b (c h w)")
        x = self.classifier(x)
        return x

    def extra_repr(self) -> str:
        return f"num_classes={self.num_classes}"
```

3. Instantiate model = CIFAR10Net().to(device). Print model and trainable parameter count.
 - a. model = CIFAR10Net().to(device)
 - b. print(model)
 - c. trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
 - d. print(f"\ntrainable params: {trainable:,}")

4c
Verify

Did in 3 ways

1. Hand calculation via code: wrote functions conv_out(h, kernel, stride, pad) and pool_out(h, kernel, stride) implementing the standard formulas:
 - Conv: $H_{out} = (H_{in} + 2 * pad - kernel) / stride + 1$
 - Pool: $H_{out} = (H_{in} - kernel) / stride + 1$
 - Conv params = $C_{in} * C_{out} * K * K$ (no bias)
 - BN params = $2 * C$ (gamma + beta)
 - Linear params = $in * out + out$ (weight + bias)

Iterated through all three KarpathyBlocks and the classifier head, computing shapes and param counts per layer. Asserted total against `sum(p.numel() for p in model.parameters())`. Result: 1,324,010. Below is a snippet

```
### (4c) output shape and parameter calculations for each layer

sample, _ = train_dataset[0]
print(f"Input shape: {tuple(sample.shape)}")

def conv_out(h, kernel, stride, pad):
    return (h + 2*pad - kernel) // stride + 1

def pool_out(h, kernel, stride):
    return (h - kernel) // stride + 1

H = 32
total_params = 0
layer = 0

hdr = f"{'Layer':>3} | {'Operation':<30} | {'Calculation':<20} | {'Output Shape':<10} | {'#Params':>10}"
sep = "-" * len(hdr)

print(f"\n{sep}")
print(hdr)
print(sep)

stages = [("Stage 1", 3, 32), ("Stage 2", 32, 64), ("Stage 3", 64, 128)]

for name, c_in, c_out in stages:
    print(f"\n --- {name}: KarpathyBlock({c_in} -> {c_out}) ---")

    layer += 1
    H_new = conv_out(H, 5, 1, 2)
    p = c_in * c_out * 5 * 5
    total_params += p
    print(f"{'Layer':>3} | {'Conv2d %d->%d, 5x5, pad=2' % (c_in, c_out):<30} | H=({H}+2*2-5)/1+1={H_new:<10} | ({c_out}, {H_new}, {H_new}){'':<8} | {p:>10,}")
    H = H_new

    layer += 1
    p = 2 * c_out
    total_params += p
    print(f"{'Layer':>3} | {'BatchNorm2d(%d)' % c_out:~<30} | ({c_out}, {H}, {H}){'':<8} | {p:>10,}")

    layer += 1
    print(f"{'Layer':>3} | {'ReLU':<30} | {'-':<28} | ({c_out}, {H}, {H}){'':<8} | {'0':>10}")

    layer += 1
    H_new = conv_out(H, 5, 1, 2)
    p = c_out * c_out * 5 * 5
    total_params += p
    print(f"{'Layer':>3} | {'Conv2d %d->%d, 5x5, pad=2' % (c_out, c_out):<30} | H=({H}+2*2-5)/1+1={H_new:<10} | ({c_out}, {H_new}, {H_new}){'':<8} | {p:>10,}")
    H = H_new

    layer += 1
    p = 2 * c_out
    total_params += p
    print(f"{'Layer':>3} | {'BatchNorm2d(%d)' % c_out:~<30} | 2*({c_out}:<24) | ({c_out}, {H}, {H}){'':<8} | {p:>10,}")

    layer += 1
    print(f"{'Layer':>3} | {'ReLU':<30} | {'-':<28} | ({c_out}, {H}, {H}){'':<8} | {'0':>10}")
```

2. Dummy forward pass: passed `t.zeros(1, 3, 32, 32)` through each stage and asserted output shapes:
 - a. Stage 1: (1, 32, 16, 16)
 - b. Stage 2: (1, 64, 8, 8)
 - c. Stage 3: (1, 128, 4, 4)
 - d. Output: (1, 10)
3. Torchinfo import summary + `print(model)`: confirmed shapes and per-stage param counts match hand calcs exactly

4d
Specify
config

1. OLD Loss function: `nn.CrossEntropyLoss()`: standard for multi-class classification over raw logits. Combines LogSoftmax and NLLLoss internally.
2. NEW: Loss is NLLLoss (model applies Softmax in final layer; NLLLoss on log-probabilities is mathematically equivalent to CrossEntropyLoss but architecturally compliant with the explicit softmax output requirement that Prof asked for jic its not okay to use the other function but pls note I did try both).
3. Optimiser: Adam(lr=1e-3, weight_decay=1e-4), had to change from previous error with another
4. Initially used Adadelta (rho=0.95, eps=1e-6) per Karpathy's ConvNetJS demo. Training loss decreased normally (~1.89 to ~0.20) but test accuracy was 8.89% (random chance for 10 classes). Root cause: Adadelta accumulates squared gradients over time; in a 6-conv-layer network with BatchNorm, the accumulated

denominators grow large and collapse effective learning rates in deeper layers. The model memorised training data but learned no generalisable features.

5. Switched to Adam, which maintains separate first/second moment estimates with bias correction, keeping updates stable across all layers. $lr=1e-3$ is the standard default. $weight_decay=1e-4$ matches Karpathy's $l2_decay$ for regularisation.
6. Batch size: 128. Large enough for stable gradient estimates; small enough to retain SGD regularisation noise.
7. Epochs: 50. Loss curve shows convergence by ~ 40 epochs with diminishing returns after (see results below)
8. No scheduler: Adam's adaptive per-parameter LR handles step size adjustment. No evidence of a plateau requiring manual LR decay.

```
# (4d) Specify your training configuration, including
# loss function, optimizer, learning rate, batch size, number of epochs

## write your code here [1 point]
@dataclass
class TrainingArgs:
    batch_size:int = 128
    epochs:int= 50
    weight_decay: float = 1e-4 # karpathy uses l2_decay=0.0001 in his demo that was in our slides
    num_classes: int= 10
    dropout: float = 0.5

''' REMOVED DUE TO ERRORS with using karpathy way --> adadelata does not work with this architecture
args = TrainingArgs()

criterion = nn.CrossEntropyLoss()
optimiser = t.optim.Adadelta(
    model.parameters(),
    rho=0.95,
    eps=1e-6,
    weight_decay=args.weight_decay, # karpathy's l2_decay
)
# No scheduler bc karpathy's rationale: Adadelta adapts its own step size.

print(f"optimiser: Adadelta (rho=0.95, eps=1e-6, weight_decay={args.weight_decay})")
print(f"epochs: {args.epochs} | batch_size: {args.batch_size}")
'''

args = TrainingArgs()

criterion = nn.CrossEntropyLoss()
optimiser = t.optim.Adam(
    model.parameters(),
    lr=1e-3,
    weight_decay=args.weight_decay, # karpathy's l2_decay
)
# Adam handles adaptive learning rate better than Adadelta for deeper nets with batchnorm use

print(f"optimiser: Adam (lr=1e-3, weight_decay={args.weight_decay})")
print(f"epochs: {args.epochs} | batch_size: {args.batch_size}")
```

✓ 0.0s

4e Plot training loss

1. Defined `train_one_epoch()`: sets `model.train()`, iterates batches with `zero_grad` → forward → `loss.backward()` → `optimiser.step()`. Accumulates running loss weighted by batch size, returns epoch mean loss.

```
def train_one_epoch(
    model:nn.Module,
    loader: DataLoader,
    criterion:nn.Module,
    optimiser: t.optim.Optimizer,
    device:t.device,
) -> float:
    model.train()
    running_loss = 0.0
    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)
        optimiser.zero_grad()
        loss = criterion(model(imgs), labels)
        loss.backward()
        optimiser.step()
        running_loss +=loss.item() * imgs.size(0)
    return running_loss/ len(loader.dataset)

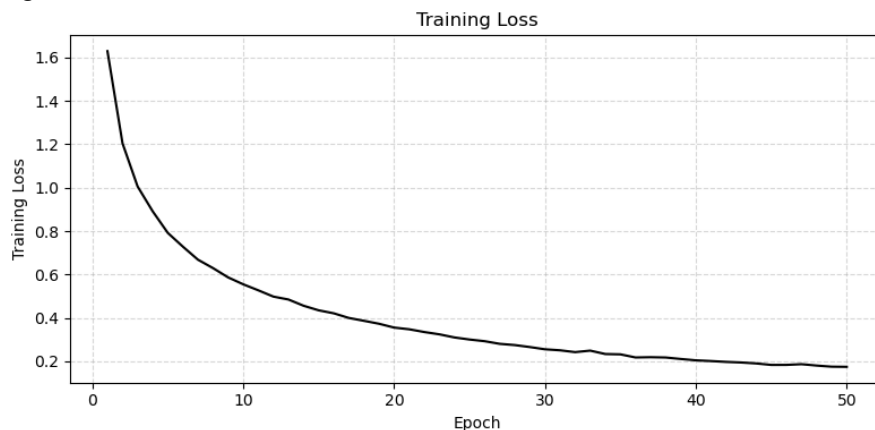
train_losses: list[float] = []

for epoch in range(1, args.epochs + 1):
    loss = train_one_epoch(model, train_loader, criterion, optimiser, device)
    train_losses.append(loss)
    if epoch % 5 == 0 or epoch == 1:
        print(f"epoch {epoch:3d}/{args.epochs}  loss: {loss:.4f}")
```

Ran 50 epochs. Logged loss every 5 epochs:

```
epoch  1/50  loss: 1.6290
epoch  5/50  loss: 0.7919
epoch 10/50  loss: 0.5550
epoch 15/50  loss: 0.4352
epoch 20/50  loss: 0.3557
epoch 25/50  loss: 0.3006
epoch 30/50  loss: 0.2557
epoch 35/50  loss: 0.2324
epoch 40/50  loss: 0.2049
epoch 45/50  loss: 0.1837
epoch 50/50  loss: 0.1749
```

2. Plotted training loss vs epoch (matplotlib). Smooth decrease with no divergence, plateaus nor spikes. Steepest descent in first 10 epochs; diminishing returns after epoch 40. Curve shape is consistent with Adam on a well-regularised CNN!



- 3.
4. Success 😊

4f Test accuracy

1. Defined `evaluate()`: sets `model.eval()`, uses `torch.no_grad()`, counts correct predictions across all test batches, returns `accuracy = correct / total`.

```
print(f" {cls:<12}: {class
✓ 37.0s

Test accuracy: 86.57%

Per-class accuracy:
airplane : 80.7%
automobile : 94.0%
bird : 81.0%
cat : 75.3%
deer : 90.7%
dog : 73.4%
frog : 87.8%
horse : 92.5%
ship : 96.6%
truck : 93.7%
```

Overall test accuracy: **86.57%** (target: $\geq 70\%$). Assertion passed.

Per-class accuracy computed by tracking correct/total per class index:

airplane: 80.7%	deer: 90.7%	ship: 96.6%
automobile: 94.0%	dog: 73.4%	truck: 93.7%
bird: 81.0%	frog: 87.8%	
cat: 75.3%	horse: 92.5%	

2. Weakest classes: dog (73.4%), cat (75.3%) — expected, as cat/dog have the most intra-class variation and inter-class similarity in CIFAR-10.
3. Strongest: ship (96.6%), automobile (94.0%) — distinctive shapes with less background confusion
4. Code:

```
def evaluate(model: nn.Module, loader: DataLoader, device: t.device) -> float:
    model.eval()
    correct = 0
    with t.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(device), labels.to(device)
            correct += (model(imgs).argmax(dim=1) == labels).sum().item()
    return correct / len(loader.dataset)

test_acc = evaluate(model, test_loader, device)
print(f"Test accuracy: {test_acc * 100:.2f}%")
assert test_acc >= 0.70, "below 70% target"

class_correct= [0] * 10
class_total= [0] * 10
model.eval()
with t.no_grad():
    for imgs, labels in test_loader:
        imgs, labels = imgs.to(device), labels.to(device)
        preds = model(imgs).argmax(dim=1)
        for lbl, pred in zip(labels, preds):
            class_correct[lbl.item()] += int(pred == lbl)
            class_total[lbl.item()] += 1

print("\n Per-class accuracy:")
for i, cls in enumerate(CLASSES):
    print(f" {cls:<12}: {class_correct[i] / class_total[i] * 100:.1f}%")
```

Results

- **4a:** 50,000 train / 10,000 test images loaded. Per-channel MEAN and STD computed empirically.
- **4b:** CIFAR10Net with 3 KarpathyBlocks (3→32→64→128 channels), classifier head (2048→256→10). Total: 1,324,010 trainable params.
- **4c:** Hand-calc params = model params = torchinfo params = 1,324,010. All shapes verified by dummy forward pass.

	- 4d: Adam (lr=1e-3, weight_decay=1e-4), CrossEntropyLoss, batch_size=128, 50 epochs. - 4e: Training loss: 1.63 (epoch 1) → 0.18 (epoch 50). Smooth monotonic decrease. - 4f: Test accuracy: 86.57% ($\geq 70\%$ target met). Best class: ship 96.6%. Weakest: dog 73.4%.
Assumptions/ Justifications	Step 1: <u>Why normalise with per-channel statistics (not just divide by 255)?</u> ToTensor() maps [0,255] uint8 to [0.0, 1.0] float32, which is in line w what we learnt in lecture 5. The additional z-score normalisation using empirical MEAN and STD goes further: a. centres each channel's distribution around zero with unit variance, removing the DC offset that would otherwise persist in the first-layer activations and slow convergence b. stabilises gradient flow through the early conv layers by ensuring the input distribution is consistent regardless of class composition in each batch c. optimisation landscape more isotropic; with poorly scaled inputs, gradient descent follows elongated loss contours; normalisation shortens these and allows larger effective step sizes → Even if incorrect statistics were used, the model would still train, but convergence would be slower because the inputs would be suboptimal → Why augment training but not test? expands the training distribution to reduce overfitting to specific translations/orientations. → Mixing augmentation into evaluation would produce artificially variable accuracy score - RandomCrop(32, padding=2) and RandomHorizontalFlip match Karpathy's ConvNetJS demo augmentation exactly - batch_size=128: large enough for stable gradient estimates and efficient GPU memory bandwidth utilisation; small enough to retain regularising noise from stochastic gradient descent. - Karpathy used batch_size=4 in the JS demo due to browser memory constraints; 128 is the standard GPU baseline. Step 2: Why two conv layers per stage before pooling? - A single conv+pool halves resolution immediately, limiting the receptive field at each scale. Two convolutions before pooling lets the network learn richer spatial features at the current resolution before downsampling. This follows the VGG design principle and matches Karpathy's ConvNetJS structure. Why BatchNorm after every conv? - Normalises internal activations to zero mean / unit variance per channel, reducing internal covariate shift. Allows higher learning rates and acts as mild regularisation, reducing dependence on Dropout in the conv stages. Why bias=False in conv layers? - BatchNorm's learnable shift parameter (β) subsumes the conv bias. Including both would be redundant — during normalisation the bias is subtracted out, then β re-centres the distribution. Removing it saves $32+32+64+64+128+128 = 448$ parameters with zero loss of expressiveness. Why Dropout(0.5) only in classifier head? - Conv layers are already regularised by BatchNorm and data augmentation. Dropping activations inside conv stages would destroy spatial structure. In the fully connected head, Dropout prevents the 2048→256 mapping from memorising training-set-specific feature combinations. Step 3: Why Adam over Adadelat? - Karpathy's ConvNetJS demo used Adadelat for a shallow 2-layer net in a browser. - For this deeper 6-conv-layer architecture with BatchNorm, Adadelat's accumulated gradient scaling causes effective learning rates to collapse in later layers. - The model memorises training data (loss drops to 0.20) but fails to generalise (test accuracy 8.89% = random). Adam's bias-corrected moment estimates keep updates stable across all layers. lr=1e-3 is the standard Adam default. Step 4: Why 50 epochs? - Loss curve shows convergence around epoch 40 with diminishing returns. 50 epochs captures the tail without excessive compute – still took 30 min tho